TRƯỜNG ĐẠI HỌC SÀI GÒN (SGU) • KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO TOÀN DIỆN PHẦN 4: ĐIỀU PHỐI HỆ THỐNG, DEMO & VẤN ĐÁP

ĐỀ TÀI: QUẢN LÝ CỬA HÀNG THỜI TRANG (OOP JAVA) • PHẦN 4

Phân công: Thành viên 4 (Trưởng nhóm) | GVHD: Thầy Trần Ngọc Minh Hiếu (tnmhieu@sgu.edu.vn)

TÀI LIỆU CHI TIẾT DÀNH CHO TRƯỞNG NHÓM & BẢO VỆ VẤN ĐÁP (ĐIỂM 10/10)

> **Phân công phụ trách:** Thành viên 4 (Trưởng nhóm / Người thuyết trình & Demo chính)

> **Mục tiêu:** Nắm toàn bộ kiến trúc điều phối hệ thống (Facade Pattern), làm chủ vòng lặp giao diện Console Menu 10 chức năng (CRUD), quy trình bán hàng - in phiếu bảo hành tự động, kịch bản demo 7 bước mượt mà và bộ câu hỏi vấn đáp ăn trọn điểm 10 của Thầy Hiếu.

1. DANH MỤC CÁC TẬP TIN (FILES) QUẢN LÝ TRONG PHẦN 4

Phần 4 gồm tổng cộng **3 tập tin mã nguồn (.java)**:

1.1. Các tập tin mã nguồn chính (Core Files) - 2 Lớp

1. `QuanLyCuaHang.java` : Lớp điều phối trung tâm (Controller / Facade Pattern) kết nối giữa giao diện và các kho dữ liệu.
2. `Main.java` : Lớp giao diện dòng lệnh Console (View & Menu Dispatcher), tiếp nhận mọi tương tác của người dùng.

1.2. Tập tin kiểm thử tự động (Testing & Verification) - 1 Lớp

3. `TestCases.java` : Bộ kịch bản kiểm thử tự động gồm **73 testcase** kiểm tra độ bền vững, đa hình, kế thừa, ngoại lệ và tính đúng đắn của toàn bộ đồ án.

2. BẢNG PHÂN LOẠI KĨ THUẬT LẬP TRÌNH: CHÍNH VS PHỤ TRỢ

Phân loại	Tên kĩ thuật	Vị trí xuất hiện trong mã nguồn	Vai trò & Ý nghĩa kỹ thuật
Kỹ thuật chính	**Mô hình Điều phối Facade (Facade Pattern)**	Lớp `QuanLyCuaHang`	Che giấu sự phức tạp của các kho dữ liệu bên dưới; gom toàn bộ `DanhSachSanPham`, `DanhSachKhachHang`, `NhaCungCap` vào 1 đầu mối duy nhất.
Kỹ thuật chính	**Điều phối giao diện Menu (Menu Dispatcher)**	Vòng lặp `while (true)` & `switch-case` trong `Main.java`	Phân luồng điều khiển 10 chức năng hệ thống; đảm bảo ứng dụng chạy liên tục cho đến khi người dùng chọn thoát.

Kỹ thuật chính	**Quy trình Thanh toán & Xuất hóa đơn**	Case 8 trong `Main.java`, phương thức `displayBill()`, `xuatHoaDonRaFile()`	Tích hợp toàn diện: Chọn nhiều mặt hàng, cộng dồn số lượng, tra cứu giảm giá VIP/Nhân viên, tính VAT 10%, in phiếu bảo hành và lưu file text.
Kỹ thuật chính	**Kiểm thử tự động (Automated Unit Testing)**	`TestCases.java` (73 testcase)	Minh chứng tính đúng đắn khoa học của đồ án trước giảng viên: Không lỗi logic, không crash ngoại lệ, 100% testcase PASS.
Kỹ thuật phụ trợ	**Chống lỗi trôi lệnh Scanner**	`scanner.nextLine().trim()` kết hợp `Integer.parseInt`	Tránh triệt để lỗi bỏ qua dòng (Skip line) khét tiếng của `scanner.nextInt()` trong Java khi nhấn phím Enter.
Kỹ thuật phụ trợ	**Ép kiểu Downcasting có điều kiện**	`if (oldSp instanceof AoKhoac) { ((AoKhoac) oldSp).setHasHat(...); }`	Ép kiểu an toàn từ lớp cha về lớp con để sửa các thuộc tính đặc thù (mũ áo, size áo, chất liệu, size giày).
Kỹ thuật phụ trợ	**Cơ chế Tự động Lưu tức thì (Auto-save)**	Hàm `saveDataIntoFile()` trong `QuanLyCuaHang.java`	Tự động ghi đè dữ liệu mới nhất xuống file text sau mỗi thao tác Thêm, Sửa, Xóa; không bao giờ mất dữ liệu.
Kỹ thuật phụ trợ	**Tìm kiếm kép (Dual Search)**	Tìm theo mã chính xác & tìm theo từ khóa tên gần đúng	Tăng trải nghiệm người dùng, giúp tra cứu nhanh chóng khi không nhớ chính xác mã sản phẩm.

3. PHÂN TÍCH CHI TIẾT TỪNG ĐOẠN CODE: CHÍNH VS PHỤ TRỢ

3.1. Lớp điều phối trung tâm `QuanLyCuaHang.java`

```
public class QuanLyCuaHang {
    private DanhSachSanPham dsSanPham;
    private DanhSachKhachHang dsKhachHang;
    private ArrayList<NhaCungCap> dsNhaCungCap;

    public QuanLyCuaHang() {
        this.dsSanPham = new DanhSachSanPham();
        this.dsKhachHang = new DanhSachKhachHang();
        this.dsNhaCungCap = new ArrayList<>();
        khoiTaoNhaCungCapMacDinh();
    }

    public void addProduct(SanPhamThoiTrang product) {
        if (dsSanPham.them(product)) {
            saveDataIntoFile();
        }
```

```
    }

    public void saveDataIntoFile() {
        dsSanPham.ghiVaoFile("products.txt");
        dsKhachHang.ghiVaoFile("users.txt");
    }

    public void loadDataFromFile(String fileName) {
        dsSanPham.docTuFile(fileName);
        dsKhachHang.docTuFile("users.txt");
    }
}
```

- **Kỹ thuật chính (Facade Pattern):**
- Gom 3 kho dữ liệu: `dsSanPham`, `dsKhachHang`, `dsNhaCungCap`.
- Hàm `main` chỉ cần tạo duy nhất 1 đối tượng `QuanLyCuaHang store = new QuanLyCuaHang()`. Mọi thao tác thêm sản phẩm, thêm khách hàng, thanh toán đều gọi qua `store`.
- **Kỹ thuật phụ trợ (Auto-save):**
- Trong `addProduct`, `removeProduct`, `updateProduct`: Bất cứ khi nào thao tác mảng trả về `true`, hệ thống tự động gọi ngay `saveDataIntoFile()`.

3.2. Kỹ thuật tiếp nhận dữ liệu an toàn trong `Main.java`

```
while (true) {
        inMenu();
        System.out.print("Nhap lua chon cua ban: ");
        String input = scanner.nextLine().trim();
        int choice = -1;
        try {
            choice = Integer.parseInt(input);
        } catch (NumberFormatException e) {
            System.out.println("Vui long nhap so hop le!");
            continue;
        }
        // ... xử lý switch case
    }
```

- **Kỹ thuật chính:** Vòng lặp Menu chính điều khiển toàn bộ ứng dụng.
- **Kỹ thuật phụ trợ (Phòng chống lỗi nhập liệu):**
- Sử dụng `scanner.nextLine().trim()` thay vì `scanner.nextInt()`: Khi dùng `nextInt()`, phím Enter (`\n`) vẫn còn lưu lại trong bộ đệm (Buffer), làm cho lệnh `nextLine()` kế tiếp bị trôi qua mà không cho người dùng kịp nhập. Cách đọc chuỗi rồi ép kiểu bằng `Integer.parseInt` loại bỏ hoàn toàn lỗi này.
- Khối `try-catch (NumberFormatException)`: Nếu người dùng lỡ tay gõ chữ cái thay vì số, chương trình bắt lỗi nhẹ nhàng và nhắc nhở chứ không bị crash đột ngột.

3.3. Chức năng Sửa toàn diện kết hợp Downcasting (Menu 4)

```
if (editOpt.equals("4")) {
        if (oldSp instanceof AoKhoac) {
            System.out.println("Ao co non khong? (1: Co / 0: Khong): ");
            boolean hasHat = scanner.nextLine().trim().equals("1");
            ((AoKhoac) oldSp).setHasHat(hasHat);
        } else if (oldSp instanceof QuanJean) {
            System.out.println("Nhap kieu dang moi: ");
```

```
            String st = scanner.nextLine().trim().replace(",", " ");
            ((QuanJean) oldSp).setStyle(st);
        } else if (oldSp instanceof AoThun) {
            System.out.println("Nhap kich co moi (S/M/L/XL): ");
            String sz = scanner.nextLine().trim();
            ((AoThun) oldSp).setSize(sz);
        } else if (oldSp instanceof GiayDep) {
            System.out.println("Nhap size giay moi (35 - 45): ");
            int sg = Integer.parseInt(scanner.nextLine().trim());
            ((GiayDep) oldSp).setSizeGiay(sg);
        }
        store.saveDataIntoFile();
        System.out.println("Da cap nhat thuoc tinh rieng thanh cong!");
    }
```

- **Kỹ thuật chính (Downcasting & Instanceof):**
- Biến `oldSp` ban đầu có kiểu khai báo là lớp cha `SanPhamThoiTrang` (chỉ có các phương thức chung như `getTenSP` , `getGiaBan`). Lớp cha không hề có các phương thức `setHasHat` hay `setSizeGiay` .
- Nhờ toán tử `instanceof` kiểm tra đúng kiểu thực tế và cú pháp ép kiểu xuống `((AoKhoac) oldSp)` , chương trình có thể gọi được các phương thức riêng biệt của từng lớp con.

3.4. Quy trình Bán hàng, Chiết khấu & In Bảo hành (Menu 8)

```
HoaDon newbill = new HoaDon(newBillCode, null);
    // Nhập sản phẩm và số lượng...
    SanPhamThoiTrang product = store.getProductByCode(pCode);
    newbill.addProduct(product, sl);

    // Nhập số điện thoại khách hàng:
    Nguoi person = store.getVIPOrStaffByPhone(sdt);
    newbill.setPerson(person);

    // In hóa đơn và lưu file:
    newbill.displayBill();
    newbill.xuatHoaDonRaFile("hoadon.txt");
```

```
// Bên trong HoaDon.java: In bảo hành tự động
    public void printAllWarrantyCards() {
        for (ChiTietHoaDon ct : chiTietList) {
            SanPhamThoiTrang sp = ct.getSanPham();
            if (sp instanceof IBaoHanh) {
                ((IBaoHanh) sp).printWarrantyCard();
            }
        }
    }
```

- **Kỹ thuật chính:**
- `newbill.setPerson(person)` : Thiết lập đối tượng thanh toán.
- Trong `displayBill()` : Tự động gọi đa hình `((IGiamGiaKhachHang) person).applyDiscount(total)` nếu khách là VIP (giảm 20%), Thân thiết (giảm 10%), hoặc Nhân viên (giảm 50%).
- `printAllWarrantyCards()` : Tự động in phiếu bảo hành cho những món đồ cài đặt `IBaoHanh` .
- `xuatHoaDonRaFile("hoadon.txt")` : Ghi lại toàn bộ nội dung hóa đơn và phiếu bảo hành vào file nhật ký.

4. KỊCH BẢN DEMO ĐỒ ÁN TRƯỚC GIẢNG VIÊN (7 BƯỚC ĐẠT ĐIỂM 10)

4. **Bước 1 (Tổng quan kiến trúc - 30 giây):** Thưa Thầy, đồ án nhóm em xây dựng hệ thống Quản lý cửa hàng thời trang với **20 lớp đối tượng và 3 interface**, tổ chức theo mô hình phân tầng: Model Sản phẩm, Model Con người, Lớp mảng danh sách đóng gói, Lớp điều phối Facade và View Console.
5. **Bước 2 (Xem danh mục hàng hóa - Phím 5):** Bấm phím 5 hiển thị toàn bộ kho hàng dạng bảng Excel, có phân cột rõ ràng và định dạng tiền tệ chuẩn Việt Nam (`1.800.000 VND`).
6. **Bước 3 (Thêm sản phẩm tự sinh mã - Phím 1):** Chọn thêm một đôi Giày dép mới. Cho Thầy thấy hệ thống **tự động sinh mã `GD002`** mà người dùng không cần gõ mã bằng tay.
7. **Bước 4 (Tìm kiếm sản phẩm - Phím 3):** Demo 2 chế độ tìm kiếm: Tìm kiếm theo mã (`AK001`) và Tìm kiếm gần đúng theo tên (`Hunter` hoặc `Polo`).
8. **Bước 5 (Sửa thông tin sản phẩm - Phím 4):** Chọn sản phẩm vừa thêm và sửa thuộc tính riêng (ví dụ sửa size giày từ 40 lên 42). Mở file `products.txt` minh chứng dữ liệu được cập nhật ngay lập tức.
9. **Bước 6 (Bán hàng đa hình & In bảo hành - Phím 8):**

- Mua 2 Áo khoác (`AK001`) và 1 Áo thun (`AT001`).
- Nhập SĐT khách hàng VIP (`0912345678`).
- Bảng hóa đơn tự động tính VAT 10%, giảm giá 20% cho VIP và **tự động in kèm Phiếu bảo hành 12 tháng** cho Áo khoác.
- Mở file `hoadon.txt` cho Thầy xem hóa đơn được lưu trữ vĩnh viễn.

10. **Bước 7 (Minh chứng kiểm thử tự động - 1 phút):** Chạy lệnh `java TestCases` trên Terminal, hiển thị kết quả **73/73 testcase vượt qua tuyệt đối (0 thất bại)**.

5. BỘ 10 CÂU HỎI VẤN ĐÁP KINH ĐIỂN CỦA THẦY HIẾU (KÈM CÂU TRẢ LỜI ĐIỂM 10)

11. **Thầy hỏi: Đồ án có đủ 15 lớp theo yêu cầu không? Kể tên?**

Trả lời: Dạ thưa Thầy, đồ án của nhóm em có tổng cộng **20 lớp và 3 interface**. Gồm 7 lớp sản phẩm thời trang, 6 lớp phân cấp con người, 4 lớp quản lý và danh sách mảng, 2 lớp hóa đơn chi tiết, 1 lớp thực thi `Main` và 3 interface nghiệp vụ.

12. **Thầy hỏi: Lớp `QuanLyCuaHang` đóng vai trò gì trong kiến trúc hướng đối tượng?**

Trả lời: Dạ thưa Thầy, `QuanLyCuaHang` áp dụng mẫu thiết kế **Facade Pattern (Bộ điều phối trung tâm)**. Nó đóng gói các kho dữ liệu rời rạc bên dưới (`DanhSachSanPham` , `DanhSachKhachHang` , `NhaCungCap`) thành một giao diện đơn giản duy nhất để lớp `Main` gọi tới, giúp giảm sự phụ thuộc (Loose Coupling) giữa giao diện người dùng và cơ sở dữ liệu.

13. **Thầy hỏi: Đa hình được thể hiện ở những vị trí nào quan trọng nhất?**

Trả lời: Thể hiện ở 3 vị trí cốt lõi: (1) Đa hình qua lớp trừu tượng `SanPhamThoiTrang` với các hàm `getThongTinRieng()` , `getStringSaveFile()` , (2) Đa hình bảo hành qua `IBaoHanh.printWarrantyCard()` , (3) Đa hình chiết khấu khách hàng/nhân viên qua `IGiamGiaKhachHang.applyDiscount()` .

14. **Thầy hỏi: Lớp mảng danh sách đối tượng trong đồ án là gì?**

Trả lời: Nhóm xây dựng 2 lớp mảng chuyên biệt là `DanhSachSanPham` và `DanhSachKhachHang` . Chúng đóng gói `ArrayList` bên trong và cung cấp trọn vẹn các phương thức nghiệp vụ CRUD (Xem, Thêm, Sửa, Xóa, Tìm kiếm) và Đọc/Ghi file text.

15. **Thầy hỏi: Tại sao trong `Main.java` nhóm không dùng `scanner.nextInt()` để đọc lựa chọn menu?**

Trả lời: Vì `scanner.nextInt()` chỉ đọc số mà để lại ký tự Enter (`\n`) trong bộ đệm bàn phím, dẫn đến các lệnh `scanner.nextLine()` tiếp theo bị trôi qua mà không nhận được dữ liệu của người dùng. Nhóm sử dụng `scanner.nextLine().trim()` kết hợp `Integer.parseInt` để triệt tiêu hoàn toàn lỗi trôi lệnh.

16. **Thầy hỏi: Kỹ thuật ép kiểu Downcasting được áp dụng ở đâu?**

Trả lời: Được áp dụng khi sửa thuộc tính riêng của sản phẩm: `((AoKhoac) oldSp).setHasHat(hasHat)` và khi in phiếu bảo hành: `((IBaoHanh) sp).printWarrantyCard()` , sau khi đã dùng `instanceof` kiểm tra an toàn.

17. **Thầy hỏi: Nhóm xử lý lỗi font tiếng Việt khi đọc/ghi file như thế nào?**

Trả lời: Nhóm sử dụng `InputStreamReader` và `OutputStreamWriter` kết hợp chỉ định tường minh bảng mã quốc tế `StandardCharsets.UTF_8` cùng luồng đệm `BufferedReader` / `BufferedWriter` , đảm bảo hiển thị chuẩn 100% tiếng Việt trên mọi hệ điều hành.

18. **Thầy hỏi: Biến static `HoaDon.tongSoHoaDon` có ý nghĩa gì?**

Trả lời: Biến static là biến thuộc về mức Lớp, chỉ lưu duy nhất 1 vùng nhớ chia sẻ cho mọi thực thể hóa đơn, tự động tăng lên sau mỗi lần xuất đơn để thống kê chính xác tổng số hóa đơn đã phục vụ trong phiên làm việc.

19. **Thầy hỏi: Khi khách hàng mua trùng một mã sản phẩm nhiều lần trong 1 đơn hàng thì hệ thống xử lý sao?**

Trả lời: Lớp `HoaDon` duyệt giỏ hàng bằng hàm tìm kiếm theo mã; nếu đã có, hệ thống gọi phương thức `ct.tangSoLuong(sl)` để cộng dồn số lượng chứ không tạo thêm dòng mới, đảm bảo tính thẩm mỹ và nghiệp vụ của hóa đơn.

20. **Thầy hỏi: Nếu ứng dụng bị mất điện tắt đột ngột thì dữ liệu có bị mất không?**

Trả lời: Dạ không, vì hệ thống áp dụng cơ chế tự động đồng bộ tức thì (Auto-save). Bất kỳ thao tác thêm, xóa, sửa sản phẩm hay người dùng đều được ghi đè ngay lập tức xuống `products.txt` và `users.txt` .